HIV Simulation Engine - Complete Model Guide

Version: 11.2 **Last Updated:** 2026-03-11 **Purpose:** Comprehensive feature documentation and external validation reference **Academic Context:** Thesis research on HIV prevention interventions — Netherlands PrEP cost-effectiveness

> **Calibration status:** In progress. Model structure is complete and validated. Epidemiological parameter values (beta, epsilon, activity rates) are placeholders pending calibration against Dutch RIVM/SHM/ATHENA data. Do not interpret outputs as calibrated estimates.

Executive Summary

This is a discrete-time stochastic population model for HIV transmission dynamics and prevention interventions, designed for the Netherlands context. The model is implemented from scratch with modern computational best practices:

- **Label-based configuration** (not index-based)
- **Dynamic dimensions** (inferred from config)
- **Flexible time resolution** (days/months/years)
- **Comprehensive demography** (fertility, life tables, sexual debut, net migration)
- **Robust validation** (mass balance, conservation checks)
- **Time-scheduled interventions** (year-varying ART/PrEP)
- **Multi-scenario framework** (policy comparisons)
- **Advanced initialization** (census-based relative weights)
- **Garnett (1994) epsilon mixing** (contact balance guaranteed)
- **Corrected two-step Binomial transmission** (no double-counting)

Table of Contents

Part I: Core Model

1. Core Architecture
2. Time System
3. Population Representation
4. Group Definitions
5. Sexual Mixing
6. HIV Transmission
7. Disease Progression
8. Diagnosis & Treatment
9. PrEP Interventions

10. Demography
11. Outputs

Part II: Advanced Features

12. Advanced Demography
13. Sexual Debut System
14. Childhood Pool
15. Net Migration
16. Validation & Conservation Checks

Part III: Intervention Infrastructure

17. Time-Scheduled Interventions
18. Multi-Scenario Framework
19. Policy Comparisons

Part IV: Configuration & Initialization

20. V10/V11 Config Format & ConfigReader
21. Population Initialization Methods
22. Extended Age Groups & Open-Ended Bins

Part V: Appendices

- Appendix A: Simulation Loop Order
- Appendix B: Mathematical Notation
- Appendix C: File Structure
- Appendix D: Configuration Hierarchy (V10/V11)
- Appendix E: Performance
- Appendix F: Limitations & Known Constraints
- Appendix G: Validation Checklist
- Appendix H: Version History

PART I: CORE MODEL

1. Core Architecture

1.1 Model Type

- **Population-based** (not individual/agent-based)
- **Discrete-time** (fixed timesteps, not continuous/event-driven)
- **Stochastic** (binomial draws) with deterministic mode option

1.2 State Representation

Population stored as **counts in a multidimensional tensor**:

```
Shape: (n_age_groups, n_orientations, n_activities, n_hiv_stages,
        n_diagnosis, n_art, n_prep)
```

All dimensions **inferred dynamically from configuration** — no hard-coded limits.

1.3 Configuration System

The model uses a **V10/V11 hierarchical JSON config** with 7 top-level keys. A `ConfigReader` class translates this into the V6-compatible flat format consumed by simulation modules. See Section 20 for details.

- **Single unified JSON file** (v11_configuration.json)
- **Label-based** (human-readable names, not numeric indices)
- **Self-documenting** (inline notes, tier metadata, bounds, sources on every parameter)
- **Hierarchical** ordered by conceptual domain (meta, run_control, population, demography, transmission_model, interventions, output)

1.4 Programming Language

- **Python 3.10+**
- **Dependencies:** NumPy, Pandas
- **No cloud services** (local execution)

2. Time System

2.1 Flexible Time Units

Supports three time units:

- **Days** (365 per year)
- **Months** (12 per year)
- **Years** (1 per year)

2.2 Configurable Timestep

Current configuration: 3-month (quarterly) timesteps.

```
"run_control": {
  "unit": {"value": "months"},
  "step_size": {"value": 3},
  "start_year": {"value": 1970},
  "end_year": {"value": 2020}
```

```
}
```

This produces `years_per_step = 0.25` (quarterly).

2.3 Rate Conversion

Annual rates automatically converted to per-timestep using the exact formula:

```
p_step = 1 - (1 - p_annual)^(years_per_step)
```

This is the `annual_to_step_rate()` function used throughout the engine. It correctly handles any timestep size.

Important: Beta (per-act transmission probability) is NOT converted this way — it is applied per contact, and contacts are already scaled by `years_per_step`. See Section 6 for details.

2.4 Number of Timesteps

```
n_steps = (end_year - start_year) × (units_per_year / step_size)
```

Example (current config): 1970–2020 (50 years), monthly units, 3-month steps → 200 timesteps.

3. Population Representation

3.1 State Tensor Dimensions

Dimension	Description	Current Values	Configurable
Age	5-year bins	20 groups: 15-19 → 65+	✅ Yes
Orientation	Sexual orientation	Female, Heterosexual_Male, MSM	✅ Yes
Activity	Sexual activity level	Low, Medium, High	✅ Yes
HIV Status	Disease stage	Susceptible, Acute, Chronic, AIDS	⚠️ Fixed order
Diagnosis	Diagnosis status	Undiagnosed, Diagnosed	❌ Fixed (2 states)
ART	Treatment status	Not on ART, On ART	❌ Fixed (2 states)
PrEP	Prevention status	None + enabled interventions	✅ Dynamic

3.2 Example State

```
A 25-year-old MSM with high sexual activity, chronic HIV,
diagnosed, on ART, not on PrEP:
```

```
population[2, 2, 2, 2, 1, 1, 0] = count
           |  |  |  |  |  |  └─ PrEP: None (index 0)
           |  |  |  |  |  └─ ART: On ART (index 1)
           |  |  |  |  └─ Diagnosis: Diagnosed (index 1)
           |  |  |  └─ HIV: Chronic (index 2)
           |  |  └─ Activity: High (index 2)
           |  └─ Orientation: MSM (index 2)
           └─ Age: 25–29 (index 2)
```

3.3 Validation

- No negative counts (enforced per-timestep)
- All values finite
- Mass balance preserved across all transitions
- Comprehensive conservation checks at initialization

4. Group Definitions

4.1 Age Groups

Current configuration: 20 five-year bins from 15-19 through 65+.

```
"age_groups": [
  {"name": "15–19", "min": 15, "max": 19},
  {"name": "20–24", "min": 20, "max": 24},
  ...
  {"name": "60–64", "min": 60, "max": 64},
  {"name": "65+",   "min": 65, "max": 110}
]
```

Open-ended `"65+"` bin acts as an accumulator for the oldest cohort. See Section 22.

4.2 Sexual Orientations

Three orientations:

- **Female** — mixes with Heterosexual_Male
- **Heterosexual_Male** — mixes with Female
- **MSM** — mixes with MSM

Mixing rules are hard-coded by orientation pair; no cross-pair contacts (e.g. no MSM-Female contacts).

4.3 Sexual Activity Groups

Three activity levels with annual contact rates:

```
"sexual_activity_groups": [
  {"name": "Low",    "contact_rate": 3.0},
  {"name": "Medium", "contact_rate": 10.0},
  {"name": "High",   "contact_rate": 50.0}
]
```

Note: These contact rate values are placeholders pending calibration.

Orientation Contact Multipliers: Scales base contact rates by orientation to reflect differential sexual activity:

```
"orientation_contact_multipliers": {
  "value": {
    "Female": 0.7,
    "Heterosexual_Male": 1.0,
    "MSM": 2.5
  }
}
```

Effective rate = base_contact_rate × orientation_multiplier. Defaults to 1.0 if not specified.

4.4 HIV Disease Stages

Fixed order (do not modify):

1. Susceptible (HIV-negative)
2. Acute (recent infection, high viral load)
3. Chronic (established infection, lower viral load)
4. AIDS (advanced disease, immune failure)

4.5 PrEP States

Dynamically built at initialization:

- Always includes `"None"` (no PrEP)
- Adds each enabled intervention by name (e.g., `"Oral_PrEP"`, `"Lenacapavir"`)

5. Sexual Mixing

5.1 Overview: Garnett (1994) Epsilon Parameterisation

V11 uses the **Garnett (1994) epsilon parameterisation** for activity-class mixing, replacing the earlier explicit mixing matrix approach. This guarantees contact balance by construction.

Two scalar parameters control assortativity:

- `epsilon_hetero` — for heterosexual mixing (Female ↔ Heterosexual_Male)

- `epsilon_msm` — for MSM mixing (MSM ↔ MSM)

```
ε = 0  → fully proportional (random) mixing across activity classes
ε = 1  → fully assortative (same activity class only)
```

Current config values are placeholders pending calibration against Dutch sexual behaviour data.

5.2 Mixing Rule

For a female of activity class `f` forming contacts with males of activity class `m`:

```
ρ(f→m) = (1 − ε) × q(m) + ε × δ(f,m)
```

Where:

- `q(m)` = proportional contact availability of male activity class `m` (weighted by population × contact rate)
- `δ(f,m)` = 1 if f = m (assortative term), 0 otherwise
- `ε` = epsilon_hetero

5.3 Partnership Pool and Contact Balance

Heterosexual (female-driven pool):

Partnership counts are set by the female side:

```
pool(f, m) = N_F(f) × c_F(f) × years_per_step × ρ(f→m)
```

The same pool value is used for both directions:

```
contacts[F, f, HM, m] = contacts[HM, m, F, f] = pool(f, m)
```

This guarantees that total female contacts = total male contacts (no double-counting), which is a fundamental demographic requirement.

MSM (symmetric averaged pool):

For MSM, both sides are computed and symmetrised:

```
contacts[MSM, i, MSM, j] = contacts[MSM, j, MSM, i] = mean(raw_i→j, raw_j→i)
```

5.4 Contact Calculation

```
total_contacts[orient, activity] =
    population[orient, activity, summed] × contact_rate × orient_multiplier × years_per_step
```

The `years_per_step` scaling means contacts accumulate linearly with time; the engine does NOT convert beta separately (beta is already per-act).

5.5 Condom Usage

Activity-specific condom effectiveness modifier (currently unused in transmission formula — see Section 6):

```
"condom_usage": {
  "Low": 0.5,
  "Medium": 0.7,
  "High": 0.9
}
```

6. HIV Transmission

6.1 Corrected Two-Step Binomial Engine

V11 uses the `transmission_corrected.py` engine implementing a **two-step Binomial formulation** that correctly handles simultaneous multiple contacts.

Step 1 — Force of infection per susceptible individual:

```
λ = 1 − ∏_{s ∈ HIV stages} (1 − β_s × ART_modifier_s)^(n_contacts_from_stage_s)
```

Where:

- `β_s` = per-act transmission probability for HIV stage `s` (read directly from config, no timestep scaling)
- `ART_modifier_s = (1 − art_transmission_reduction)` if proportion on ART > 0
- `n_contacts_from_stage_s` = contacts with HIV+ partners in stage `s`

Step 2 — New infections:

```
new_infections ~ Binomial(n_susceptible, λ × PrEP_modifier)
```

Where `PrEP_modifier = 1 − (proportion_on_PrEP × PrEP_efficacy)` .

6.2 Beta Interpretation

Beta is a **per-contact (per-act) transmission probability**. Contacts are already scaled by `years_per_step` in the mixing engine, so beta is applied directly without any timestep conversion.

Current placeholder values (pending calibration — see CALIBRATION_NOTES.md):

Stage	Current config	Literature target (Boily 2009)
Acute	0.015	~0.00658 (M→F vaginal)
Chronic	0.004	~0.00072 (M→F vaginal)
AIDS	0.008	~0.00553 (M→F vaginal)

Note: Current values are incorrect placeholders. The AIDS/Chronic ratio in the current config (~2×) is in the wrong direction; literature supports ~7.7×. See BACKLOG item DESIGN-3 and TRANSMISSION_BETA_DISCUSSION.md.

6.3 Direction-Specific Betas

Currently a single beta is applied per stage regardless of transmission direction (M→F vs F→M vs MSM). Literature suggests M→F and F→M differ by approximately 0.5× (Boily 2009). Implementation of direction-specific betas is tracked in BACKLOG-12.

6.4 ART Effect on Transmission

Treatment as Prevention (TasP / U=U):

```
"art_transmission_reduction": {"value": 0.96}
```

96% reduction in per-act transmission probability when source partner is on ART.

6.5 Infection Assignment

New infections:

- Move from Susceptible → Acute
- Enter as Undiagnosed
- Not on ART
- Lose PrEP status (if any)
- Distributed across age groups proportionally to susceptible pool

7. Disease Progression

7.1 HIV Stages

Progression pathway:

```
Susceptible → Acute → Chronic → AIDS → (death)
```

7.2 Progression Rates

Annual probabilities (current placeholders):

```
"progression": {
  "acute_to_chronic": {"value": 4.0},
  "chronic_to_aids": {"value": 0.05},
  "chronic_to_aids_on_art": {"value": 0.005}
}
```

Converted to per-timestep using `annual_to_step_rate()`. Note that `acute_to_chronic = 4.0` means ~98% progression per year, reflecting the short (~10–12 week) acute phase.

7.3 ART-Dependent Progression

Chronic → AIDS progression is ART-dependent:

- **Off ART:** uses `chronic_to_aids` rate (~5%/year)
- **On ART:** uses `chronic_to_aids_on_art` rate (~0.5%/year, 10× slower)

Acute → Chronic progression is ART-independent (ART is rarely initiated in the acute phase).

7.4 Implementation

Binomial draws per stage:

```
progressions ~ Binomial(population[stage], progression_rate_per_step)
```

Counts move from current stage to next stage each timestep.

8. Diagnosis & Treatment

8.1 Diagnosis Cascade

Undiagnosed HIV+ → Diagnosed

```
"diagnosis": {
  "annual_rate": {"value": 0.15}
}
```

Only HIV+ individuals can be diagnosed. Rate is constant in current implementation; time-scheduled diagnosis rates are a future calibration target.

8.2 ART Initiation

Diagnosed, not on ART → On ART

Controlled by time-scheduled rates (V5+):

```
"art": {
  "schedule": {
    "initiation": {
      "value": {
        "1996": 0.10,
        "2000": 0.30,
        "2005": 0.50,
        "2010": 0.70,
        "2015": 0.80,
        "2020": 0.85
      }
    }
  }
}
```

ART introduced in 1996; zero uptake before that year.

8.3 ART Dropout

On ART → Diagnosed, not on ART

```
"dropout_rate": {"value": 0.05}
```

5% annual probability of discontinuing ART (Netherlands ATHENA cohort estimate ~95% retention at 1 year).

8.4 ART Effects

Transmission reduction (TasP):

```
effective_beta = beta × (1 − art_transmission_reduction × proportion_on_ART)
```

Mortality reduction:

```
hiv_mortality_on_ART = hiv_mortality × (1 − art_mortality_reduction)
```

Current `art_mortality_reduction = 0.90` (90% reduction in HIV mortality on ART).

Progression reduction: Chronic → AIDS rate is 10× lower on ART (see Section 7.3).

9. PrEP Interventions

9.1 Supported Interventions

- **Oral PrEP** (`oral_prep`) — daily pills, 86% efficacy
- **Lenacapavir** (`lenacapavir`) — injectable, 6-month interval *(future)*
- **Extensible** — any new intervention added to the config is automatically registered

9.2 PrEP Configuration (V10/V11 format)

In the V10 config, PrEP interventions live under `interventions.oral_prep` :

```
"oral_prep": {
  "enabled": {"value": false},
  "uptake_rate": {"value": 0.05},
  "dropout_rate": {"value": 0.20},
  "advanced": {
    "efficacy": {"value": 0.86},
    "dosing_interval": {"value": 3, "unit": "months"},
    "uptake_modifier_by_activity": {
      "value": {"Low": 0.5, "Medium": 1.0, "High": 1.5}
    }
  }
}
```

`ConfigReader.interventions()` renames `oral_prep` → `Oral_PrEP` and promotes `advanced.efficacy` to the top level before the engine sees it.

9.3 Time-Scheduled PrEP Rates

When enabling PrEP with time-varying rates, add a `prep_schedules` block at the top level of the V10 config:

```
"prep_schedules": {
  "interpolation": {"value": "stepwise"},
  "Oral_PrEP": {
    "initiation": {
      "schedule": {"2017": 0.10, "2020": 0.20}
    },
    "discontinuation": {
      "schedule": {"2017": 0.20}
    }
  }
}
```

Important: If `art_schedule` is present (which it is in the current config), the engine creates a `ScheduleManager` and routes ALL PrEP transition rates through it. If `prep_schedules` is absent, the ScheduleManager returns 0 for PrEP initiation regardless of `uptake_rate` in the config. Always provide `prep_schedules` when enabling PrEP. See `config_reader.py` line ~445 for the forwarding mechanism.

9.4 PrEP Transitions

Susceptible only can use PrEP:

- None → PrEP (uptake, per timestep)
- PrEP → None (dropout, per timestep)

HIV infection removes PrEP: new Acute infections always have PrEP = None.

9.5 PrEP Effect on Transmission

Applied in the two-step Binomial:

```
PrEP_modifier = 1 − (proportion_on_PrEP × efficacy)
```

Applied to the susceptible individual's infection probability, not to the infectious partner's beta.

10. Demography

10.1 Demography Modes

Three modes supported:

Closed: No births, aging and mortality only. Population declines over time.

Fixed Net Addition: Constant annual entrants added to entry age group.

Fertility Table (current): ASFR-based births with year-specific multipliers and childhood pool. Most realistic.

10.2 Mortality

HIV Mortality (stage-specific annual rates):

```
"mortality": {
  "acute":   {"value": 0.005},
  "chronic": {"value": 0.010},
  "aids":    {"value": 0.250}
}
```

Current values are placeholders. Literature-based targets: AIDS ~0.40–0.50/year in pre-ART era (e.g., Morgan 2002), lower chronic/acute. See CALIBRATION_NOTES.md.

Background Mortality: Applied via age/sex-specific life tables (Netherlands CBS data). Life table mortality is applied to all individuals in addition to HIV-specific mortality.

10.3 Aging

Default: Fractional aging (smooth, continuous)

Each timestep, a fraction of each age bin progresses to the next:

```
annual_outflow_fraction = 1 / bin_width_years = 0.20 (for 5-year bins)
p_step = 1 - (1 - 0.20)^(years_per_step)
```

This produces smooth, continuous demographic flows with no mortality spikes. The oldest bin (65+) accumulates without further aging.

Legacy: Discrete aging (not recommended — produces mortality spikes every 5 years). Retained for backward compatibility.

Aging method is configurable:

```
"demography": {
  "advanced": {
    "aging_method": {"value": "fractional"}
  }
}
```

10.4 Fertility

Age-specific fertility rates (ASFR) derived from Netherlands CBS data with year-specific multipliers to capture TFR decline:

```
"fertility": {
  "asfr_by_age_limit": {
    "value": [
      {"age_limit": 20, "birth_rate": 0.0028},
      {"age_limit": 25, "birth_rate": 0.0850},
      ...
    ]
  },
  "advanced": {
    "year_multipliers": {
      "value": {"1970": 1.0, "1980": 0.98, "1990": 0.95, ...}
    }
  }
}
```

Births go directly into the childhood pool (0-4 bin).

11. Outputs

11.1 File Outputs

Per-run and aggregated CSV files written to `outputs/` :

```
outputs/
├── run_001.csv
├── run_002.csv
└── aggregated.csv       (mean + Q05/Q50/Q95 across runs)
```

11.2 Timeseries Columns

Column	Description
step	Timestep number (0-indexed)
year	Calendar year (floating point)
new_infections	HIV infections this timestep
prevalence	HIV+ / sexually active population
incidence	New infections / susceptible
hiv_deaths	HIV-related deaths
background_deaths	Non-HIV deaths
total_population	Total population (active + childhood)
sexually_active_population	Adults in transmission system
childhood_population	Children (0-14)
births	Births this timestep
net_migration	Net migration this timestep (V9)
prep_coverage_Oral_PrEP	PrEP coverage
prep_coverage_Lenacapavir	PrEP coverage

11.3 Terminal Summary

```
Year       Population HIV Prev %    New Inf   HIV Deaths    BG Deaths
1970       12,749,613       9.91      7,441        5,497       16,490
...
2020       11,543,417      19.52     20,349        5,395       34,591

PrEP Coverage (final year):
  Oral_PrEP: 0.00%
```

PART II: ADVANCED FEATURES

12. Advanced Demography

12.1 Life Tables

Age/sex-specific background mortality derived from Netherlands CBS life tables:

```
"life_table": {
  "enabled": {"value": true},
  "survival_by_age_limit": {
    "value": [
      {"age_limit": 1,   "male_survival": 0.99622, "female_survival": 0.99687},
      {"age_limit": 5,   "male_survival": 0.99913, "female_survival": 0.99943},
      ...
      {"age_limit": 110, "male_survival": 0.00001, "female_survival": 0.00005}
    ]
  }
}
```

Mortality per step = `1 - survival_annual^(years_per_step)`, applied to all individuals (HIV+ receive both background and HIV-specific mortality additively).

12.2 Population Adjustments

Optional exogenous scaling to match external population targets during calibration:

```
"population_adjustments": {
  "enabled": {"value": true},
  "mode": "year_specific",
  "year_multipliers": {
    "value": {"1970": 1.0, "1980": 1.02, ...}
  }
}
```

Applied at the start of each timestep before transmission.

13. Sexual Debut System

13.1 Overview

Sexual debut is a **continuous per-timestep transition** from the oldest childhood bin (10-14) into the youngest adult bin (15-19), ensuring smooth demographic inflow.

13.2 Implementation

Sexual debut is handled via fractional aging of the childhood pool — the outflow from the 10-14 bin each timestep naturally feeds into the 15-19 bin. This means debut is implicit in the fractional aging mechanism, not a separate

module.

At debut, individuals are assigned:

- **Orientation:** Males split into MSM (`p_msm = 0.10`) and Heterosexual_Male; females all Female
- **Activity:** All new debutants enter as Low activity (configurable)
- **HIV status:** All Susceptible, Undiagnosed, no ART, no PrEP

13.3 Configuration

```
"sexual_debut": {
  "debut_age": {"value": 15},
  "p_msm": {"value": 0.10}
}
```

14. Childhood Pool

14.1 Overview

Individuals aged 0-14 are tracked in a separate childhood pool (outside the HIV transmission tensor). This supports complete demographic lifecycle modelling.

Childhood pool structure:

```
Shape: (n_childhood_bins, n_sexes)
```

Current bins: 0-4, 5-9, 10-14. Sexes: female, male.

14.2 Flows

Inflows: Births (from fertility module), immigrating children (if migration enabled)

Outflows: Sexual debut (10-14 → 15-19 adult bin), childhood mortality, aging (0-4 → 5-9 → 10-14)

Not modelled in childhood: HIV status (all assumed susceptible), orientation, activity, ART, PrEP.

14.3 Initialization

With `relative_weights` initialization, childhood counts are automatically derived from the same `age_sex_distribution` used for adults. The sexual debut boundary (age 15) splits the distribution.

15. Net Migration (V9)

15.1 Overview

V9 introduced a net migration module (`migration.py`) supporting realistic Dutch immigration/emigration flows. Migration is applied once per timestep.

15.2 Configuration

```
"demography": {
  "migration": {
    "enabled": {"value": true},
    "quantity": {
      "mode": {"value": "year_specific"},
      "year_specific": {
        "value": {
          "1970": 20000,
          "1980": 25000,
          ...
        }
      }
    },
    "profile": {
      "mode": {"value": "working_age"},
      "working_age": {
        "value": {
          "age_range": [20, 49],
          "male_fraction": 0.55
        }
      }
    }
  }
}
```

15.3 Mass Balance

Net migration is tracked separately in the global mass balance equation (added in V9):

```
N(t+1) = N(t) + births − HIV_deaths − background_deaths + net_migration
```

Migrants enter as susceptible, undiagnosed, no ART, no PrEP, with activity assigned by profile weights.

16. Validation & Conservation Checks

16.1 Global Population Mass Balance

Enforced every timestep:

```
N(t+1) = N(t) + births − HIV_deaths − background_deaths − childhood_deaths
         + adjustment_change + net_migration
```

Tolerance: ±2 persons (rounding). Raises `AssertionError` on failure.

16.2 PrEP Mass Balance

Verified every timestep: total population on PrEP must equal sum across all PrEP state dimensions.

16.3 ART Mass Balance

Verified every timestep: ART transitions (initiation, dropout, deaths on ART) must balance.

16.4 Initialization Conservation Checks (6 checks)

Run once at initialization:

1. Total population = target population (±0.001%)
2. Sexually active = expected (ages ≥15)
3. Childhood pool = expected (ages <15)
4. Female orientation = expected
5. Heterosexual_Male orientation = expected
6. MSM orientation = expected
7. Activity distribution correct
8. No negative counts

PART III: INTERVENTION INFRASTRUCTURE

17. Time-Scheduled Interventions

17.1 Overview

Year-varying intervention rates are managed by `ScheduleManager` (schedule_utils.py). When `art_schedule` or `prep_schedules` is present in the config, the ScheduleManager is activated and **takes priority over fixed rates** in the intervention config.

Applies to:

- ART initiation and dropout rates
- PrEP uptake and dropout rates

17.2 ART Schedules

Defined in V10 config under `interventions.art.schedule`. ConfigReader translates this to the V6 `art_schedule` block consumed by ScheduleManager:

```
  "art": {
    "schedule": {
      "interpolation": {"value": "stepwise"},
      "initiation": {
        "value": {
          "1996": 0.10,
          "2000": 0.30,
          "2005": 0.50,
          "2010": 0.70,
          "2015": 0.80
        }
      },
      "dropout": {
        "value": {
          "1996": 0.15,
          "2010": 0.05
        }
      }
    }
  }
```

Interpolation modes:

- `"stepwise"` : Carry-forward (rate stays constant until next defined year)
- `"linear"` : Linear interpolation between years

Rate is 0 before the first defined year (ART not yet introduced).

17.3 PrEP Schedules

Defined as top-level `prep_schedules` in the V10 config (forwarded by ConfigReader):

```
  "prep_schedules": {
    "interpolation": {"value": "stepwise"},
    "Oral_PrEP": {
      "initiation":      {"schedule": {"2017": 0.10}},
      "discontinuation": {"schedule": {"2017": 0.20}}
    }
  }
```

Note: Key must be `"Oral_PrEP"` (V6 display name), not `"oral_prep"` (V10 config key). ConfigReader handles the translation.

18. Multi-Scenario Framework

18.1 Overview

`run_scenarios.py` and `scenario_executor.py` support running multiple named scenarios from a base config with parameter overrides.

18.2 Configuration

Scenarios are defined in the base config:

```
"scenarios": [
  {
    "name": "baseline",
    "description": "Status quo",
    "overrides": {}
  },
  {
    "name": "prep_scaleup",
    "description": "Aggressive PrEP scale-up",
    "overrides": {
      "interventions": {
        "oral_prep": {
          "enabled": {"value": true},
          "uptake_rate": {"value": 0.30}
        }
      },
      "prep_schedules": { ... }
    }
  }
]
```

Overrides use **nested dict deep-merge** (not dot-notation). Each override recursively merges onto a deep copy of the base config.

18.3 Running Scenarios

```
python3 run_scenarios.py configs/v11_configuration.json
```

Each scenario writes to its own output subdirectory.

19. Policy Comparisons

19.1 Baseline Freeze

For HTA-style counterfactual analysis, the `state_snapshot.py` module supports saving and reloading population state at a given calendar year:

1. Run baseline 1970–2020 (historical calibration period)
2. Snapshot population state at 2020
3. Branch into policy scenarios from the frozen 2020 state
4. All scenarios share identical pre-2020 history → clean counterfactual

19.2 Example Use Case

Research question: What is the impact of lenacapavir PrEP vs continued oral PrEP, 2023–2030?

1. Calibrate and run baseline 1970–2020
2. Freeze state at 2020
3. Branch scenarios: status quo, oral PrEP expanded, lenacapavir introduced
4. Compare new infections averted 2023–2030

PART IV: CONFIGURATION & INITIALIZATION

20. V10/V11 Config Format & ConfigReader

20.1 V10/V11 Config Structure

The V10/V11 config uses 7 top-level keys:

Key	Contents
`meta`	Model name, version, academic context
`run_control`	Start/end year, step size, n_runs, seed
`population`	Initialization method, size, HIV seed
`demography`	Mortality, fertility, migration, aging
`transmission_model`	Groups, mixing, disease parameters
`interventions`	ART schedule, oral_prep, lenacapavir
`output`	File settings, output directory

20.2 ConfigReader

`ConfigReader` (config_reader.py) is the bridge between V10/V11 raw JSON and the V6-compatible flat format expected by simulation modules.

Key translations performed:

- `run_control.*` → `time.*` and `simulation_controls.*`
- `transmission_model.groups.*` → `groups.*`
- `transmission_model.mixing.*` → `mixing.*`
- `transmission_model.disease.*` → `disease.*`
- `interventions.oral_prep` → `interventions["Oral_PrEP"]` (with `advanced.efficacy` promoted)
- `interventions.art.schedule` → `art_schedule`
- `prep_schedules` → forwarded as-is to V6 dict

`as_v6_dict()` produces the full V6-compatible dict used internally. Both V10 and legacy V6 configs are supported.

20.3 Parameter Tiers

Every parameter in V11 carries tier metadata:

- **Tier 1 (fixed):** Directly from authoritative sources, not varied (e.g., time periods, census population size)
- **Tier 2 (estimated):** Will be calibrated against Dutch surveillance data
- **Tier 3 (uncertain):** Literature-based plausible ranges, wide uncertainty

21. Population Initialization Methods

21.1 Two Methods Available

Method 1: Direct Counts — specify exact integer counts per orientation × age × activity cell. Used for small stylized models.

Method 2: Relative Weights (current, census-based) — provide age/sex distribution weights and a target population size. Engine allocates automatically.

21.2 Relative Weights Algorithm

Step 1: Normalize age × sex distribution weights (sum to 1.0)

Step 2: Apply sexual debut boundary — ages <15 → childhood pool; ages ≥15 → sexually active tensor

Step 3: Split males by orientation — `p_msm` fraction → MSM; remainder → Heterosexual_Male

Step 4: Apply activity distribution — each orientation × age group split by Low/Medium/High proportions

Step 5: Seed initial HIV — `initial_hiv_prevalence` fraction placed in Chronic stage (Undiagnosed, no ART)

Step 6: Run 6 conservation checks (see Section 16.4)

21.3 Current Initialization

```
Target population: 12,713,815 (Netherlands 1970 census)
MSM proportion: 10% of eligible males
Activity: 60% Low / 30% Medium / 10% High
Initial HIV: 0.0 (placeholder — not yet seeded with calibrated value)
```

22. Extended Age Groups & Open-Ended Bins

22.1 Current Age Structure

20 five-year bins: 15-19, 20-24, ..., 60-64, 65+.

The 65+ bin uses `"min": 65, "max": 110` and accumulates the oldest cohort without further aging.

22.2 Open-Ended Bin Parsing

Three formats supported:

- **Closed:** `"15–19"` → min_age=15, max_age=19
- **Open-ended:** `"65+"` → min_age=65 (no upper bound in practice)
- **Single:** `"15"` → min_age=15

22.3 Life Table Alignment

Life tables must extend to cover all age bins. The current table extends to age 110, covering the 65+ open-ended bin.

PART V: APPENDICES

Appendix A: Simulation Loop Order

V11 Timestep Sequence (from run_v3.py `step_simulation()`):

1. Population adjustments (if enabled)
2. Contacts & mixing (Garnett epsilon, female-driven pool)
3. Transmission (corrected two-step Binomial)
4. Disease progression (Acute→Chronic, Chronic→AIDS, ART-dependent)
5. Diagnosis
6. ART transitions (time-scheduled via ScheduleManager)
7. PrEP transitions (time-scheduled via ScheduleManager)
8. Births → childhood pool (fertility table)
9. Childhood mortality
10. HIV mortality + adult background mortality
11. Aging (fractional) + sexual debut
12. Net migration
13. Global mass balance validation
14. PrEP mass balance validation
15. ART mass balance validation
16. Record outputs

Order is critical. Transmission before progression means newly infected individuals are in Acute stage immediately, which is intentional. Mortality after aging prevents double-counting.

Appendix B: Mathematical Notation

Rate Conversion (annual → per-step):

```
p_step = 1 − (1 − p_annual)^(years_per_step)
```

Contact Count per Timestep:

```
c(o, a) = N(o, a) × rate(a) × mult(o) × years_per_step
```

Garnett Mixing Proportion (heterosexual):

```
ρ(f→m) = (1 − ε) × q(m) + ε × δ(f,m)
q(m) = [N(HM,m) × c(HM,m)] / Σ_m'[N(HM,m') × c(HM,m')]
```

Partnership Pool (female-driven):

```
pool(f,m) = N(F,f) × c(F,f) × ρ(f→m)
contacts[F,f,HM,m] = contacts[HM,m,F,f] = pool(f,m)
```

Force of Infection (two-step Binomial, per susceptible):

```
λ = 1 − ∏_s (1 − β_s × ART_mod_s)^(n_contacts_with_stage_s / N_susceptible)
```

New Infections:

```
I_new ~ Binomial(N_susceptible, λ × PrEP_modifier)
```

PrEP Modifier:

```
PrEP_modifier = 1 − (c_PrEP × ε_PrEP)
```

Where `c_PrEP` = proportion of susceptibles on PrEP, `ε_PrEP` = efficacy.

Fertility:

```
B = Σ_a (N_a^F × ASFR_a × years_per_step × m_year)
```

Mass Balance:

```
N(t+1) = N(t) + B(t) - D_HIV(t) - D_BG(t) - D_child(t) + ΔAdj(t) + Mig(t)
```

Appendix C: File Structure

```
v11/
├── run_v3.py                      # Main simulation engine & entry point
├── run_scenarios.py               # Multi-scenario runner
├── scenario_executor.py           # Scenario execution logic
├── engine_v3.py                   # TimeSystem, GroupSystem, PopulationState
├── transmission_corrected.py      # Corrected two-step Binomial engine
├── config_reader.py               # V10→V6 config translation (ConfigReader)
├── config_utils.py                # extract_value() and helpers
├── demography.py                  # DemographyModule (fertility, mortality, aging)
├── migration.py                   # Net migration module (V9)
├── schedule_utils.py              # ScheduleManager, TimeSchedule (V5)
├── output_writer.py               # CSV/Excel output + terminal summaries
├── state_snapshot.py              # Population state save/restore
├── validation.py                  # ConfigValidator
├── regression_test.py             # Regression test harness
│
├── configs/
│   ├── v11_configuration.json     # Primary canonical config (V11.2)
│   ├── v10_configuration.json     # Previous version reference
│   └── v6_golden_reference.json   # Legacy V6 format reference
│
├── outputs/                       # Generated simulation outputs
│   └── run_001.csv, aggregated.csv
│
├── tests/                         # Unit and integration tests
│   ├── test_config_reader.py
│   ├── test_contact_balance.py
│   ├── test_art_dependent_progression.py
│   ├── test_timestep_invariance.py
│   └── ...
│
├── diagnostics/                   # Diagnostic scripts
│   ├── convergence_test.py
│   └── diagnose_mortality_rates.py
│
└── documentation/
    ├── BACKLOG.md                 # Tracked design/implementation tasks
    ├── DOCUMENTATION_INDEX.md     # Index of all documentation
    ├── guides/
    │   ├── README_MODEL_GUIDE.md  # This document
    │   └── V5_QUICK_START.md
    ├── model_notes/
    │   ├── CALIBRATION_NOTES.md   # Parameter calibration working doc
    │   ├── TRANSMISSION_BETA_DISCUSSION.md
    │   ├── CONFIGURABLE_PARAMETERS.md
    │   └── ...
    ├── patches/                   # Individual feature/fix documentation
    ├── validation/                # Validation test results
    └── investigations/            # Root cause analyses
```

Appendix D: Configuration Hierarchy (V10/V11)

The current config (`v11_configuration.json`) uses the V10/V11 hierarchical structure:

```
v11_configuration.json
├── meta
│   ├── model_name, version, author
│   └── model_notes (calibration_status, academic_framing)
├── run_control
│   ├── start_year, end_year, step_size, unit
│   ├── n_runs, seed, deterministic_mode
│   └── output_prefix
├── population
│   ├── initialization_method ("relative_weights")
│   ├── target_population
│   ├── p_msm
│   ├── activity_distribution
│   ├── age_sex_distribution
│   ├── childhood_pool ("auto")
│   └── initial_hiv_prevalence (currently 0.0 — placeholder)
├── demography
│   ├── enabled, mode ("fertility_table")
│   ├── life_table (survival_by_age_limit)
│   ├── fertility (asfr_by_age_limit, year_multipliers)
│   ├── migration (enabled, quantity, profile)
│   ├── childhood (bins, mortality)
│   ├── sexual_debut (debut_age, p_msm)
│   └── advanced (aging_method, background_mortality, population_adjustments)
├── transmission_model
│   ├── groups
│   │   ├── orientations, age_groups, sexual_activity_groups, hiv_stages
│   │   └── advanced.orientation_contact_multipliers
│   ├── mixing
│   │   ├── epsilon_hetero, epsilon_msm   ← V11 Garnett parameterisation
│   │   └── advanced.condom_usage
│   └── disease
│       ├── transmission (beta_acute, beta_chronic, beta_aids)
│       ├── progression (acute_to_chronic, chronic_to_aids, chronic_to_aids_on_art)
│       ├── mortality (acute, chronic, aids)
│       ├── diagnosis (annual_rate)
│       └── advanced (art_transmission_reduction, art_mortality_reduction)
├── interventions
│   ├── art
│   │   └── schedule (interpolation, initiation, dropout)
│   ├── oral_prep
│   │   ├── enabled, uptake_rate, dropout_rate
│   │   ├── efficacy (top-level copy for engine compatibility)
│   │   └── advanced (efficacy, dosing_interval, uptake_modifier_by_activity)
│   └── lenacapavir (same structure)
├── output
│   ├── enabled, output_dir, prefix, format
│   └── advanced (write_state_counts, summary_years)
└── [top-level] prep_schedules      ← added when enabling PrEP with schedules
    └── Oral_PrEP (initiation, discontinuation schedules)
```

Note on ConfigReader: When `simulate_v3()` loads this config, `ConfigReader.as_v6_dict()` flattens the hierarchy into the V6-compatible format used by all simulation modules. Direct config key access in the engine always uses V6 paths (e.g., `config["disease"]["transmission"]["beta_chronic"]`), not V10 paths.

Appendix E: Performance

Typical performance (modern laptop CPU, 200 timesteps, 12.7M population):

Metric	Value
Single 50-year run (200 steps)	~2–4 seconds
100-seed ensemble	~200–400 seconds
Memory usage	~100–200 MB
Config validation + initialization	<1 second

Optimizations:

- NumPy vectorised operations throughout (no Python loops over individuals)
- Efficient 7-dimensional tensor state
- Contacts computed as matrix operations

Appendix F: Limitations & Known Constraints

By Design (intentional simplifications documented as limitations):

1. **Discrete timesteps** — not continuous-time; timestep must be short enough for accurate approximation (quarterly is appropriate for HIV)
2. **Group-based mixing** — not explicit partnerships; memoryless (no sustained partnership memory)
3. **Homogeneous within groups** — individuals in the same state are treated as identical
4. **Memoryless sexual contacts** — no sustained partnerships, concurrency not modelled; appropriate for a population-level model
5. **Single transmission beta per stage** — no direction-specificity (M→F vs F→M) yet; BACKLOG-12
6. **No MSM distinct beta** — MSM contacts use a multiplier over vaginal betas; BACKLOG-12/13
7. **No superinfection** — not modelled
8. **No viral load stratification** — HIV stage is a proxy
9. **No STI cofactors** — not modelled
10. **No resistance** — ART assumed fully effective; resistance not tracked
11. **No geographic stratification** — single well-mixed population
12. **Fixed orientation** — no MSM↔Heterosexual transitions

Technical limitations:

1. Fixed HIV stage order (Susceptible, Acute, Chronic, AIDS) — cannot reorder without code changes
2. Diagnosis and ART dimensions are binary (2 states each) — cannot add intermediate states without engine changes

These are intentional simplifications appropriate for population-level cost-effectiveness modelling for a thesis context.

Appendix G: Validation Checklist

Configuration:

- [ ] Config loads without errors
- [ ] `epsilon_hetero` and `epsilon_msm` defined (V11 mode)
- [ ] All probabilities in [0, 1]
- [ ] Age groups cover sexually active range (≥15)
- [ ] Life tables extend to maximum age (110)
- [ ] Fertility rates defined for reproductive ages
- [ ] ART schedule starts at 1996 (Netherlands ART introduction)

Initialization:

- [ ] Population initialises without errors
- [ ] All 6 conservation checks pass
- [ ] Total population matches target (12,713,815)
- [ ] MSM proportion ~5% of total (10% of males)
- [ ] Childhood pool populated correctly
- [ ] Initial HIV seed set > 0 if epidemic dynamics required

Simulation:

- [ ] No crashes during 200-step run
- [ ] Population counts never negative
- [ ] Global mass balance preserved every step
- [ ] PrEP mass balance preserved (if PrEP enabled)
- [ ] ART mass balance preserved
- [ ] Deterministic mode reproducible (same output for same seed)

Outputs:

- [ ] Output files created in `outputs/`
- [ ] Terminal summary displays
- [ ] Prevalence and incidence move in expected direction
- [ ] ART introduction visible as inflection in prevalence curve post-1996
- [ ] PrEP coverage > 0% (if `prep_schedules` provided and `oral_prep.enabled = true`)

Appendix H: Version History

V1.0 (2026)

- Basic model with yearly timesteps
- Fixed dimensions, index-based configuration

V2.0 (2026)

- Dynamic configuration, flexible time system
- Label-based groups, PrEP interventions

V3.0 (2026)

- Robust CSV/Excel outputs
- Demography modules, life tables
- Comprehensive validation

V4.0 (2026)

- Sexual debut system (continuous hazard from childhood pool)
- Childhood pool (0-14, outside HIV transmission tensor)
- Flush-on-aging guardrails
- Enhanced fertility integration

V4.2

- Flush-on-aging implementation
- Mass balance conservation checks

V4.3

- Enhanced guardrails and validation patches

V5.0 (2026)

- Time-scheduled interventions (ART, PrEP via ScheduleManager)
- Multi-scenario framework
- Policy comparisons (baseline freeze & reuse via state_snapshot)
- Population initialization: relative weights (census-based)
- Extended age groups with open-ended bins (65+)

V5.1

- Fractional aging (smooth continuous aging, replaces discrete 5-year shifts as default)
- Fixes discrete aging time-invariance bug

V6.0

- V6 config restructure: flat key format with inline notes on all parameters
- Parameter tier system introduced (fixed / estimated / uncertain)

V9.0

- Net migration module (migration.py)
- Year-specific migration volumes and age/sex profiles
- Migration term added to global mass balance

V10.0 (2026)

- V10 config restructure: 7 top-level keys, hierarchical `advanced` blocks
- ConfigReader introduced: translates V10 → V6 for backward compatibility
- 20 age bins (65+ expanded from single bin); contact balance fix
- Corrected two-step Binomial transmission engine (transmission_corrected.py)
- Replaces V6 double-counting bug in transmission

V11.0 (2026)

- Garnett (1994) epsilon parameterisation for activity mixing
  - Replaces explicit mixing matrices with two scalars (`epsilon_hetero` , `epsilon_msm`)
  - Female-driven partnership pool for heterosexual contacts
  - MSM symmetric averaged pool
  - Contact balance guaranteed by construction
- Orientation contact multipliers (orientation-specific scaling of base contact rates)

V11.2 (2026) — **CURRENT**

- Parameter tier metadata on all epidemiological parameters (tier, source, bounds, distribution)
- `prep_schedules` forwarding added to ConfigReader (latent bug fix — PrEP now works when `art_schedule` is present)
- Calibration notes and transmission beta discussion documents added

Quick Start

To run the simulation:

```
cd v11/
python3 run_v3.py configs/v11_configuration.json
```

For calibration work:

1. Set `initial_hiv_prevalence > 0` in the population config
2. Use `relative_weights` initialization (already configured)
3. Adjust `epsilon_hetero` , `epsilon_msm` , and activity contact rates to match incidence/prevalence targets
4. Enable time-scheduled ART (already configured from 1996)

5. Enable all validation patches (active by default)
6. Compare outputs against RIVM/SHM/ATHENA Dutch surveillance targets (see CALIBRATION_NOTES.md)

For PrEP intervention analysis:

1. Calibrate baseline 1970–2020 first
2. Snapshot population state at 2020 (`state_snapshot.py`)
3. Add `prep_schedules` to config for Dutch PrEP program timeline (2017+)
4. Set `oral_prep.enabled = true`
5. Run counterfactual scenarios and compare infections averted

End of Model Guide

For calibration parameter targets and strategy, see `documentation/model_notes/CALIBRATION_NOTES.md` . For detailed beta parameter discussion, see `documentation/model_notes/TRANSMISSION_BETA_DISCUSSION.md` . For open design decisions, see `documentation/BACKLOG.md` . For all documentation, see `documentation/DOCUMENTATION_INDEX.md` .